Эталоны и отступы в обнаружении SYN-flood

Понятие эталона. Отступы и классификация объектов качества

Как объяснить решение детектора и найти случаи для перепроверки

Три части доклада: 5 + 5 + 5

Объяснение метода → выполнение задания → применение в работе ИБ

Критерий	Что показываем	Где
Теория · 5 баллов	Эталон, отступ, мягкий зазор SVM	Слайды 3–7
Задание · 5 баллов	Выбор эталонов, расчёты, пограничные окна	Слайды 8–14
Применение · 5 баллов	Сигнал детектора и действия аналитика	Слайды 15–18

Подробные расчёты — в Google Colab; на слайдах — смысл для ИБ

SYN-flood: почему страдает сервер

Теория · сначала разберём сетевую проблему

Обычное подключение

Клиент просит соединение → сервер отвечает → клиент подтверждает. Можно работать.

SYN-flood

Много запросов на соединение остаются без завершения. Очередь может заполниться.

В результате новые легитимные клиенты могут не подключиться к сервису.

Много пользователей и много SYN — ещё не доказательство атаки

Что видит наш детектор

Одно наблюдение — 10 секунд трафика к выбранному IP и TCP-порту

Что измеряем	Как читаем	Зачем это нужно
Входящие SYN/с	Сколько запросов на соединение	Оценить интенсивность
Незавершённые рукопожатия, %	Сколько попыток не завершилось за 3 с	Оценить характер нагрузки

Смотрим на оба признака вместе. В коде: SYN/с делим на 1000, проценты — на 100, чтобы сравнивать окна в выбранном масштабе.

Нужна видимость обоих направлений. Повторы SYN учитываем в скорости, но не как новые попытки.

Эталон — типичный представитель класса

Это опорный пример для сравнения наблюдений

Эталон нормы

Окно, похожее на другие подтверждённые нормальные окна

Эталон SYN-flood

Окно, похожее на другие подтверждённые окна этой атаки

Выбираем медоид: реальный объект с наименьшей суммой расстояний до остальных объектов своего класса.

Эталон показывает типичное поведение; он не описывает все возможные случаи

Отступ: насколько близка смена решения

Чем ближе окно к границе, тем легче модели «передумать»

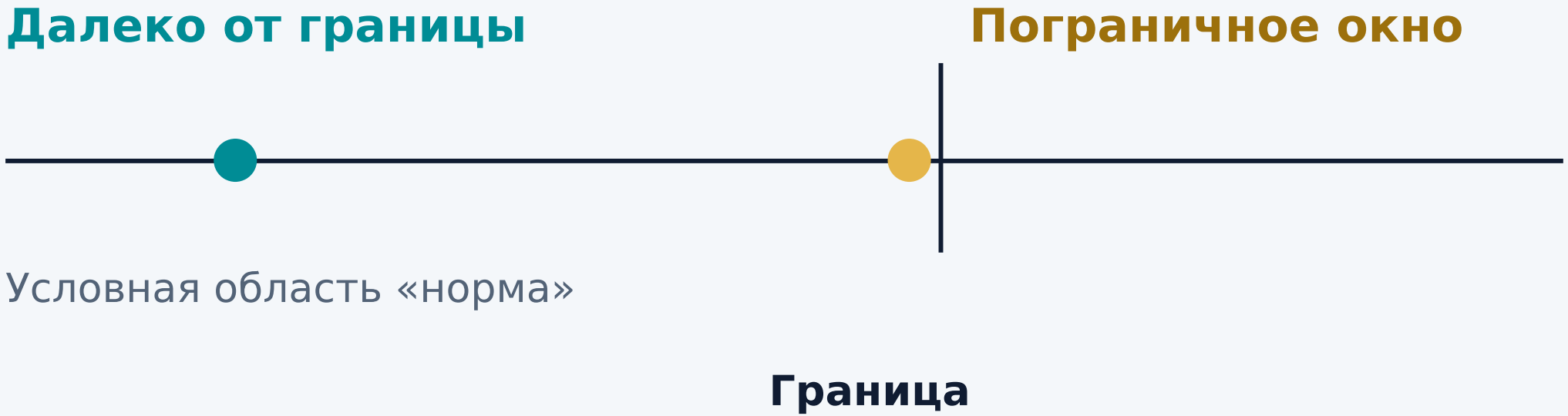


Схема: здесь показана идея расстояния, а не один из двух сетевых признаков.

Это запас сохранения ответа модели, а не вероятность атаки

SVM учится проводить границу

SVM — алгоритм, который разделяет два класса по обучающим примерам

Что делает модель

Смотрит на все обучающие окна и строит разделяющее правило

Что означает мягкий зазор

Допускает трудные примеры рядом с границей и нарушения, вместо требования идеального разделения

Параметр C регулирует штраф за нарушения. В демонстрации C = 1. Опорные векторы — обучающие точки, которые участвуют в построении границы.

Эталоны объясняют типичные примеры; SVM обучается по всей выборке

Наш учебный эксперимент

Задание · все расчёты воспроизводятся в одном ноутбуке Google Colab

20

окон для обучения
10 нормальных + 10 SYN-flood

8

отдельных проверочных окон
Есть типичные случаи и ошибки

Данные придуманы для объяснения метода. Реальная атака не запускается.

Эти восемь примеров не доказывают качество детектора на реальной сети

Шаг 1. Выбираем эталоны

Медоиды найдены кодом по 20 обучающим объектам

Класс	Окно	Входящие SYN/с	Не завершены
Нормальный трафик	N05	600	15%
SYN-flood	A06	1300	65%

Код сравнивает окна внутри каждого класса и выбирает представителя с наименьшей суммой расстояний.

Эталоны выбираем на обучении. Проверочные окна в этот выбор не попадают

Шаг 2. Обучаем SVM в Google Colab

Открываем ноутбук → выбираем «Выполнить все» → читаем таблицы

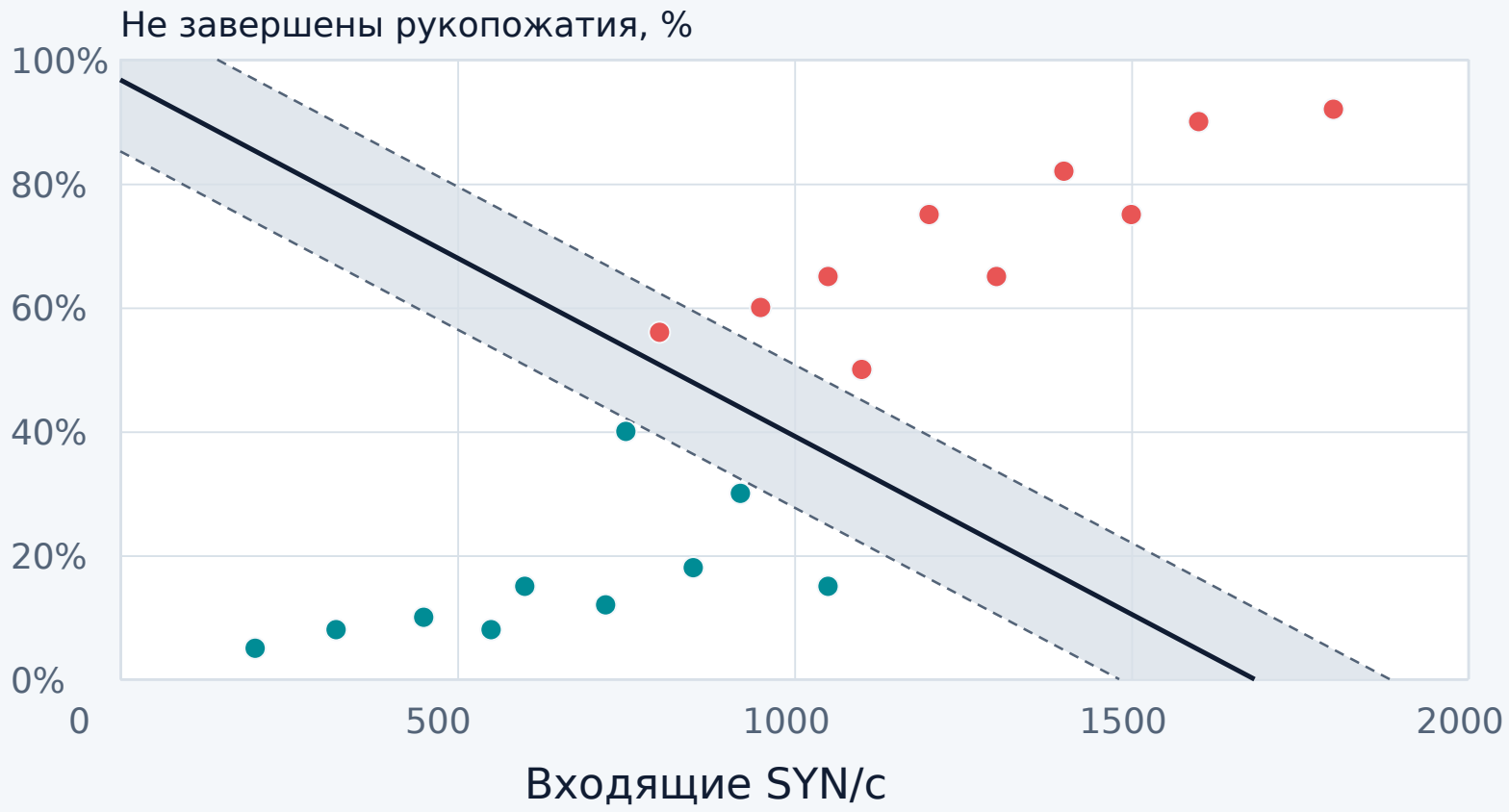
```
model = SVC(kernel="linear", C=1, tol=1e-10, shrinking=False)
model.fit(X_train, y_train)
f = model.decision_function(X_check)
distance = np.abs(f) / np.linalg.norm(model.coef_)
```

distance — расстояние до границы. Меньше значение → ближе смена текущего ответа при изменении измерений.

В ноутбуке есть данные, пояснения к каждому шагу и полные расчёты

Как модель разделяет трафик

Цвет точки — учебный класс; серая полоса — пограничные решения



Норма

ниже границы

SYN-flood

выше границы

Показаны 20 обучающих окон; полоса перепроверки: расстояние < 0,10.

Шаг 3. Сравниваем решения и отступы

Четыре из восьми проверок; истинный класс известен только в учебном примере

ID	SYN/с; не завершены	На самом деле	Модель	Запас[1]
T01	420; 9%	Норма	Норма	0,551
T02	1550; 88%	Атака	Атака	0,697
T04	900; 40%	Атака	Норма	0,043
T05	1000; 42%	Норма	Атака	0,024

[1] Запас — расстояние до границы в масштабе признаков, не процент вероятности.

Модель может ошибаться. В двух выделенных случаях запас мал

Две ошибки с разными последствиями

Именно эти случаи важно разобрать специалисту ИБ

T04 · пропуск атаки

Был SYN-flood → модель сказала «норма»
Риск: вовремя не заметить нарушение доступности сервиса

T05 · ложная тревога

Был нормальный трафик → модель сказала «атака»
Риск: помешать легитимным клиентам

Двух признаков может быть недостаточно: потери пакетов и необычная легитимная нагрузка осложняют решение.

В реальной работе правильный ответ устанавливается дополнительной проверкой

Шаг 4. Находим пограничные окна

Учебное правило: расстояние до границы меньше 0,10 → перепроверка



T03, T04, T05, T06

Повод проверить решение
даже при прогнозе «норма»

Большой запас тоже не гарантирует правильность

Порог 0,10 выбран для демонстрации. Приоритет реального инцидента зависит также от ущерба.

Применение в ИБ: сигнал для аналитика

Трафик → признаки за 10 секунд → модель и отступ → проверка в SOC

Что сказала модель	Отступ	Что делаем
Похоже на SYN-flood	Любой	Создаём сигнал о подозрении на атаку
Похоже на норму	Меньше 0,10	Дополнительно проверяем решение
Похоже на норму	От 0,10	Продолжаем обычное наблюдение

SOC — команда мониторинга и реагирования на события ИБ.
Модель помогает выделить наблюдения для проверки.

Это предлагаемая схема работы учебного прототипа; в рабочую сеть он не внедрялся.

Практическая демонстрация в Colab

Функция получает только измерения. Истинную метку мы ей не передаём

```
triage(syn_per_second=900, unfinished_percent=40)
triage(syn_per_second=1550, unfinished_percent=88)
```

Ввод: SYN/с; %	Прогноз	Сигнал атаки	Пограничное
900; 40%	Норма	Нет	Да
1550; 88%	SYN-flood	Да	Нет

«Не пограничное» не означает «безопасное»: прогноз атаки всё равно даёт сигнал

Что делает аналитик после сигнала

Двух признаков недостаточно для решения об автоматической блокировке

1. **Проверяет наблюдение**
   Видны ли ответы сервера? Нет ли потерь и неполного захвата?

2. **Сопоставляет с состоянием сервиса**
   Очередь соединений, доступность, журналы и соседние окна

3. **Выясняет контекст**
   Наплыв клиентов, изменения сервиса, TCP-прокси или балансировщик

4. **При подтверждении согласует защиту**
   Проверяет SYN cookies и правила ограничения нагрузки

Реальных журналов в этом эксперименте нет; перечислены проверки для рабочей среды.

Что нужно проверить перед внедрением

Практическое применение требует отдельной проверки на реальном трафике

Данные своей среды

Подтверждённые метки; датчик видит оба направления

Разделение по времени и сеансам

Не смешивать соседние окна одного сеанса между обучением и проверкой

Настройка на отдельной части данных

Выбрать C и порог перепроверки с учётом цены ошибок

Пробный режим без блокировок

Считать пропуски, ложные тревоги и объём работы аналитиков

CICDDoS2019 — возможный источник PCAP для следующего этапа; здесь он не использован.

Как объяснить результат по трём критериям

План защиты, соответствующий системе оценивания

Критерий	Что можно показать и объяснить
Теория · 5 баллов	Эталон — типичный образец; отступ — запас до смены ответа
Задание · 5 баллов	Два эталона, SVM, расчёт 8 окон и пограничные T03–T06
Применение · 5 баллов	Colab-демо, сигнал для SOC и проверка двух видов ошибок

Отступ помогает проверить решение модели; он не заменяет контекст ИБ

Источники

Первоисточники и официальные документы; ссылки активны

[1] Cortes & Vapnik (1995)

Support-vector networks

[2] Stanford CS229, A. Ng

Functional / geometric margin; soft margin

[3] Schubert & Rousseeuw (2019)

Faster k-Medoids Clustering

[4] Eddy (2007), RFC 4987

TCP SYN Flooding Attacks and Common Mitigations

[5] scikit-learn: документация

SVC, decision_function, масштабирование

[6] Moosavi-Dezfooli et al. (CVPR 2016)

DeepFool: расстояние до границы

[7] UNB / CICDDoS2019

Данные для следующего этапа проверки

[8] NIST SP 800-94

Проверка сигналов IDS и работа с ошибками